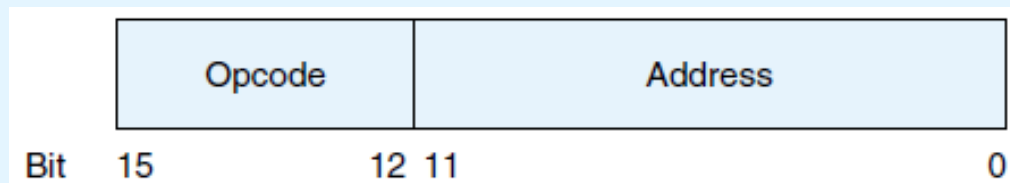


“The Essentials of Computer Organization and Architecture”

Unit 3 – Exercises and Solutions

MD1 Decipher the following MARIE machine language instructions (write the assembly language equivalent):

- i) 0010000000000111
- ii) 10010000000001011
- iii) 00110000000001001

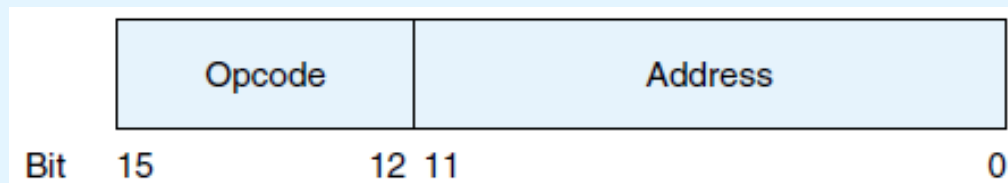


Instruction Number		Instruction
Bin	Hex	
0001	1	Load X
0010	2	Store X
0011	3	Add X
0100	4	Subt X
0101	5	Input
0110	6	Output
0111	7	Halt
1000	8	Skipcond
1001	9	Jump X

MD1 Decipher the following MARIE machine language instructions (write the assembly language equivalent):

Solution:

- i) 0010000000000111 -> **0010** 000000000111 -> **Store** 000000000111 -> **Store 007**
- ii) 1001000000001011 -> **1001** 000000001011 -> **Jump** 000000001011 -> **Jump 00B**
- iii) 0011000000001001 -> **0011** 000000001001 -> **Add** 000000001001 -> **Add 009**



Instruction Number		Instruction
Bin	Hex	
0001	1	Load X
0010	2	Store X
0011	3	Add X
0100	4	Subt X
0101	5	Input
0110	6	Output
0111	7	Halt
1000	8	Skipcond
1001	9	Jump X

MAT1 a) Assemble the following program, and
b) provide a trace for its first 4 instructions:

Address	Label	Instruction
100		Load A
101		Add One
102		Jump S1
103	S2,	Add One
104		Store A
105		Halt
106	S1,	Add A
107		Jump S2
108	A,	HEX 0023
109	One,	HEX 0001

Instruction Number		Instruction
Bin	Hex	
0001	1	Load X
0010	2	Store X
0011	3	Add X
0100	4	Subt X
0101	5	Input
0110	6	Output
0111	7	Halt
1000	8	Skipcond
1001	9	Jump X

MAT1 a) Assemble the following program, and
b) provide a trace for its first 4 instructions:

Solution (assemblage: 1st stage):

Address	Label	Instruction	1st Stage
100		Load A	1 A
101		Add One	3 One
102		Jump S1	9 S1
103	S2,	Add One	3 One
104		Store A	2 A
105		Halt	7000
106	S1,	Add A	3 A
107		Jump S2	9 S2
108	A,	HEX 0023	
109	One,	HEX 0001	

Symbol Table

A	108
One	109
S1	106
S2	103

MAT1 a) Assemble the following program, and
b) provide a trace for its first 4 instructions:

Solution (assemblage - 2nd stage):

Address	Label	Instruction	1st Stage	2nd Stage
100		Load A	1 A	1108
101		Add One	3 One	3109
102		Jump S1	9 S1	9106
103	S2,	Add One	3 One	3109
104		Store A	2 A	2108
105		Halt	7000	7000
106	S1,	Add A	3 A	3108
107		Jump S2	9 S2	9103
108	A,	HEX 0023		0023
109	One,	HEX 0001		0001

Symbol Table

A	108
One	109
S1	106
S2	103

MAT1 a) Assemble the following program, and b) provide a trace for its first 4 instructions:

Solution (trace):

Instruction			2nd Stage			
100		Load A	1108	Trace:		
101		Add One	3109			
102		Jump S1	9106			
103	S2,	Add One	3109			
104		Store A	2108			
105		Halt	7000			
106	S1,	Add A	3108	Load A Add One Jump S1 Add A		
107		Jump S2	9103			
108	A,	HEX 0023	0023			
109	One,	HEX 0001	0001			

Symbol Table

A	108
One	109
S1	106
S2	103

Trace (compact representation):

	Load A	Add One	Jump S1	Add A
MAR:	100,108	101,109	102,106	106,108
MBR:	1108,0023	3109,0001	9106	3108,0023
PC:	100,101	102	103,106	107
IR:	1108	3109	9106	3108
AC:	0023	0024	0024	0047
blue : final value				

MAT2 Solve again exercise MAT1, now assuming the program begins at memory address 200_{16} .

MAT3 a) Assemble the following program, and
b) provide a trace for its first 4 instructions:

ORG 100

```

If,      100 Load    X      /Load the first value
          101 Subt     Y      /Subtract the value of Y, store result in AC
          102 Skipcond 400    /If AC=0 (X=Y), skip the next instruction
                               /Note: This is Skipcond 01 in the book
Then,    103 Jump     Else    /Jump to Else part if AC is not equal to 0
          104 Load    X      /Reload X so it can be doubled
          105 Add      X      /Double X
          106 Store     X      /Store the new v
          107 Jump     Endif   /Skip over the f
Else,    108 Load    Y      /Start the else
          109 Subt     X      /Subtract X from
          10A Store     Y      /Store Y-X in Y
Endif,   10B Halt      /Terminate progr
X,       10C Dec       12     /Assume these va
Y,       10D Dec       20
  
```

Instruction Number		Instruction
Bin	Hex	
0001	1	Load X
0010	2	Store X
0011	3	Add X
0100	4	Subt X
0101	5	Input
0110	6	Output
0111	7	Halt
1000	8	Skipcond
1001	9	Jump X

MAT3 a) Assemble the following program, and b) provide a trace for its first 4 instructions:

ORG 100

```
If,      100 Load      X      /Load the first value
          101 Subt      Y      /Subtract the value of Y, store result in AC
          102 Skipcond  400    /If AC=0 (X=Y), skip the next instruction
                                   /Note: This is Skipcond 01 in the book
          103 Jump      Else   /Jump to Else part if AC is not equal to 0
Then,     104 Load      X      /Reload X so it can be doubled
          105 Add       X      /Double X
          106 Store     X      /Store the new value
          107 Jump      Endif  /Skip over the false, or else, part to end of if
Else,     108 Load      Y      /Start the else part by loading Y
          109 Subt      X      /Subtract X from Y
          10A Store     Y      /Store Y-X in Y
Endif,    10B Halt       /Terminate program (it doesn't do much!)
X,        10C Dec        12     /Assume these values for X and Y
Y,        10D Dec        20
```

2nd Stage

```
100: 1 10C
101: 4 10D
102: 8 400
103: 9 108
104: 1 10C
105: 3 10C
106: 2 10C
107: 9 10B
108: 1 10D
109: 4 10C
10A: 2 10D
10B: 7 000
10C: 000C
10D: 0014
```

**Solution
(assemblage):**

Conversions:

12 (10) = **C** (16)

20 (10) =
0001 0100 (2) = **14** (16)

Symbol Table

```
If      100
X       10C
Y       10D
Else    108
Then    104
Endif   10B
```

MAT3 a) Assemble the following program, and
b) provide a trace for its first 4 instructions:

ORG 100

```
If,      100 Load      X      /Load the first value
          101 Subt       Y      /Subtract the value of Y, store result in AC
          102 Skipcond   400    /If AC=0 (X=Y), skip the next instruction
                                   /Note: This is Skipcond 01 in the book
          103 Jump       Else    /Jump to Else part if AC is not equal to 0
Then,     104 Load      X      /Reload X so it can be doubled
          105 Add        X      /Double X
          106 Store      X      /Store the new value
          107 Jump       Endif   /Skip over the false, or else, part to end of if
Else,     108 Load      Y      /Start the else part by loading Y
          109 Subt       X      /Subtract X from Y
          10A Store      Y      /Store Y-X in Y
Endif,    10B Halt        /Terminate program (it doesn't do much!)
X,        10C Dec         12    /Assume these values for X and Y
Y,        10D Dec         20
```

2nd Stage

100: 1 10C
101: 4 10D
102: 8 400
103: 9 108
104: 1 10C
105: 3 10C
106: 2 10C
107: 9 10B
108: 1 10D
109: 4 10C
10A: 2 10D
10B: 7 000
10C: 000**C**
10D: 00**14**

Solution
(trace):

Trace (compact representation)

	Load X	Subt Y	Skipcond 400	Jump Else
MAR:	100, 10C	101, 10D	102, 400	103,108
MBR:	110C, 000C	410D, 0014	8400	9108
PC:	100, 101	102	103	104,108
IR:	110C	410D	8400	9108
AC:	000C	[000C-0014=FFF8]	FFF8	FFF8

MAT4 Solve again exercise MAT3, now assuming the program begins at memory address 400_{16} .

MAT5 a) Assemble the following program, and
b) provide a trace for its first 2 instructions:

```

ORG 100
100 Load X /Load the first number t
101 Store Temp /Use Temp as a parameter
102 JnS Subr /Store return address, j
103 Store X /Store first number, dou
104 Load Y /Load the second number
105 Store Temp /Use Temp as a parameter
106 JnS Subr /Store return address, j
107 Store Y /Store second number, dc
108 Halt /End program
X, 109 Dec 20
Y, 10A Dec 48
Temp, 10B Dec 0
Subr, 10C Hex 0 /Store return address he
10D Clear /Clear AC as it was modi
10E Load Temp /Actual subroutine to dc
10F Add Temp /AC now hold double the
110 JumpI Subr /Return to calling code
END
```

Instruction Number		Instruction
Bin	Hex	
0001	1	Load X
0010	2	Store X
0011	3	Add X
0100	4	Subt X
0101	5	Input
0110	6	Output
0111	7	Halt
1000	8	Skipcond
1001	9	Jump X

Instruction Number (hex)	Instruction
0	JnS X
A	Clear
B	AddI X
C	JumpI X
D	LoadI X
E	StoreI X

MAT5 a) Assemble the following program, and b) provide a trace for its first 2 instructions:

ORG 100

```

100  Load      X      /Load the first number to be doubled
101  Store     Temp   /Use Temp as a parameter to pass value to Subr
102  Jns       Subr   /Store return address, jump to procedure
103  Store     X      /Store first number, doubled
104  Load     Y      /Load the second number to be doubled
105  Store     Temp   /Use Temp as a parameter to pass value to Subr
106  Jns       Subr   /Store return address, jump to procedure
107  Store     Y      /Store second number, doubled
108  Halt
X,   109  Dec     20
Y,   10A  Dec     48
Temp, 10B  Dec     0
Subr, 10C  Hex     0      /Store return address here
10D  Clear
10E  Load     Temp   /Actual subroutine to double numbers
10F  Add       Temp   /AC now hold double the value of Temp
110  JumpI     Subr   /Return to calling code
END

```

2nd Stage

100:	1	109
101:	2	10B
102:	0	10C
103:	2	109
104:	1	10A
105:	2	10B
106:	0	10C
107:	2	10A
108:	7	000
109:	00	14
10A:	00	30
10B:	0000	
10C:	0000	
10D:	A	000
10E:	1	10B
10F:	3	10B
110:	C	10C

**Solution
(assemblage):**

Conversions:

20 (10) =
 0001 0100 (2) = **14** (16)

 48 (10) =
 0011 0000 (2) = **30** (16)

Symbol Table

X	109
Temp	10B
Subr	10C
Y	10A

MAT5 a) Assemble the following program, and
b) provide a trace for its first 2 instructions:

ORG 100

```
100 Load X /Load the first number to be doubled
101 Store Temp /Use Temp as a parameter to pass value to Subr
102 Jns Subr /Store return address, jump to procedure
103 Store X /Store first number, doubled
104 Load Y /Load the second number to be doubled
105 Store Temp /Use Temp as a parameter to pass value to Subr
106 Jns Subr /Store return address, jump to procedure
107 Store Y /Store second number, doubled
108 Halt /End program
X, 109 Dec 20
Y, 10A Dec 48
Temp, 10B Dec 0
Subr, 10C Hex 0 /Store return address here
10D Clear /Clear AC as it was modified by Jns
10E Load Temp /Actual subroutine to double numbers
10F Add Temp /AC now hold double the value of Temp
110 JumpI Subr /Return to calling code
END
```

2nd Stage

100: 1 109
101: 2 10B
102: 0 10C
103: 2 109
104: 1 10A
105: 2 10B
106: 0 10C
107: 2 10A
108: 7 000
109: 0014
10A: 0030
10B: 0000
10C: 0000
10D: A 000
10E: 1 10B
10F: 3 10B
110: C 10C

Solution
(trace):

Trace (compact representation)

	Load X	Store Temp
MAR:	100, 109	101, 10B
MBR:	1109, 0014	210B, 0014
PC:	100, 101	101, 102
IR:	1109	210B
AC:	0014	0014

MAT5 a) Assemble the following program, and
b) provide a trace for its first 2 instructions:

```

100 Load X /Load the first number to be doubled
101 Store Temp /Use Temp as a parameter to pass value to Subr
102 Jns Load X
103 Store
104 Load
105 Store
106 Jns
107 Store
108 Halt
X, 109 Dec
Y, 10A Dec
Temp, 10B Dec
Subr, 10C Hex
10D Clear
10E Load
10F Add
110 JumpI
END

```

**Solution
(trace):**

Step	RTN	PC	IR	MAR	MBR	AC
(initial values)		100	-----	-----	-----	-----
Fetch	MAR ← PC	100	-----	100	-----	-----
	IR ← M[MAR]	100	1109	100	-----	-----
	PC ← PC + 1	101	1109	100	-----	-----
Decode	MAR ← IR[11-0]	101	1109	109	-----	-----
	(decode IR[15-12])	101	1109	109	-----	-----
Get operand	MBR ← M[MAR]	101	1109	109	0014	-----
Execute	AC ← MBR	101	1109	109	0014	0014

Step	RTN	PC	IR	MAR	MBR	AC
(initial values)		101	1109	109	0014	0014
Fetch	MAR ← PC	101	1109	101	0014	0014
	IR ← M[MAR]	101	210B	101	0014	0014
	PC ← PC + 1	102	210B	101	0014	0014
Decode	MAR ← IR[11-0]	102	210B	10B	0014	0014
	(decode IR[15-12])	102	210B	10B	0014	0014
Get operand	not necessary	102	210B	10B	0014	0014
Execute	MBR ← AC	102	210B	10B	0014	0014
(changes Temp)	M[MAR] ← MBR	102	210B	10B	0014	0014

(text book representation)

MAT6 Consider again the program of exercise MAT5, but assume the program begins at address 300_{16} .

1. Assemble the program under these new conditions and present:
 - a) The symbol table resulting from the 1st assembly stage (fill the table by the order in which the symbols are found in the code during the assembly process);
 - b) The listing of opcodes (in hexadecimal) and symbols that result from the 1st assembly stage;
 - c) The listing with the final result of the assembly process, in hexadecimal.

MAT6 Consider again the program of exercise MAT5, but assume the program begins at address 300₁₆.

1. Assemble the program under these new conditions and present:

a) The symbol table resulting from the 1st assembly stage (fill the table by the order in which the symbols are found in the code during the assembly process);

Solution:

Symbol Table

X	309
Temp	30B
Subr	30C
Y	30A

MAT6 Consider again the program of exercise MAT5, but assume the program begins at address 300₁₆.

1. Assemble the program under these new conditions and present:

b) The listing of opcodes (in hexadecimal) and symbols that result from the 1st assembly stage;

Solution:

Notes:

- the column with memory addresses is not part of the listing requested
- the content of memory words 309 to 30C is absent once they are variables

```
300: 1 X
301: 2 Temp
302: 0 Subr
303: 2 X
304: 1 Y
305: 2 Temp
306: 0 Subr
307: 2 Y
308: 7 000
30D: A 000
30E: 1 Temp
30F: 3 Temp
310: C Subr
```

MAT6 Consider again the program of exercise MAT5, but assume the program begins at address 300₁₆.

1. Assemble the program under these new conditions and present:

c) The listing with the final result of the assembly process, in hexadecimal.

Solution:

Note:

- the column with memory addresses is not part of the listing requested

```
300: 1309
301: 230B
302: 030C
303: 2309
304: 130A
305: 230B
306: 030C
307: 230A
308: 7000
309: 0014
30A: 0030
30B: 0000
30C: 0000
30D: A000
30E: 130B
30F: 330B
310: C30C
```

MAT6 Consider again the program of exercise MAT5, but assume the program begins at address 300_{16} .

2. Based on the answer to question 1, trace the first four instructions executed by the program and present, in the table bellow, each instruction (by the execution order) and the final value of the the registers MAR, MBR, PC, IR and AC, right after the execution of that instruction.

	Instruction 1:	Instruction 2:	Instruction 3:	Instruction 4:
MAR				
MBR				
PC				
IR				
AC				

MAT6 Consider again the program of exercise MAT5, but assume the program begins at address 300₁₆.

2. Based on the answer to question 1, trace the first four instructions executed by the program and present, in the table bellow, each instruction (by the execution order) and the final value of the the registers MAR, MBR, PC, IR and AC, right after the execution of that instruction.

Solution:

	Instruction 1: Load X	Instrução 2: Store Temp	Instruction 3: JnS Subr	Instruction 4: Clear
MAR	309	30B	30C	000
MBR	0014	0014	030C	A000
PC	301	302	30D	30E
IR	1309	230B	030C	A000
AC	0014	0014	030D	0000

MC1 Write the following code segment in MARIE assembly language:

```
int X=20, Y=10;  
if (X > 1) {  
    Y = X + X;  
    X = 0;  
}  
Y = Y + 1;
```

Instruction Number		Instruction	Meaning
Bin	Hex		
0001	1	Load X	Load the contents of address X into AC.
0010	2	Store X	Store the contents of AC at address X.
0011	3	Add X	Add the contents of address X to AC and store the result in AC.
0100	4	Subt X	Subtract the contents of address X from AC and store the result in AC.
0101	5	Input	Input a value from the keyboard into AC.
0110	6	Output	Output the value in AC to the display.
0111	7	Halt	Terminate the program.
1000	8	Skipcond	Skip the next instruction on condition.
1001	9	Jump X	Load the value of X into PC.

MC1 Write the following code segment in MARIE assembly language:

```
int X=20, Y=10;
if (X > 1) {
    Y = X + X;
    X = 0;
}
Y = Y + 1;
```

Solution:

```
If,      Load    X      / Load X
          Subt     One    / Subtract 1, store result in AC
          Skipcond 800    / If AC>0 (X>1), skip the next instruction
          Jump     Endif  / Jump to Endif if X is not greater than 1

Then,     Load    X      / Reload X so it can be doubled
          Add      X      / Double X
          Store    Y      / Y = X + X

          Clear    / Move 0 into AC
          Store    X      / Set X to 0

Endif,    Load    Y      / Load Y into AC
          Add      One    / Add 1 to Y
          Store    Y      / Y = Y + 1

          Halt        / Terminate program

X,        Dec      20     / Storage for X (value 20 given in problem)
Y,        Dec      10     / Storage for Y (value 10 given in problem)
One,      Dec      1      / Constant 1
```

```
Load Zero
Store X
...
Zero, Dec 0
```

```
Load X
Subt X
Store X
```


MC2 Write the following code segment in MARIE assembly language:

```
int X = 1;  
while (X < 10)  
    X = X + 1;
```

Instruction Number		Instruction	Meaning
Bin	Hex		
0001	1	Load X	Load the contents of address X into AC.
0010	2	Store X	Store the contents of AC at address X.
0011	3	Add X	Add the contents of address X to AC and store the result in AC.
0100	4	Subt X	Subtract the contents of address X from AC and store the result in AC.
0101	5	Input	Input a value from the keyboard into AC.
0110	6	Output	Output the value in AC to the display.
0111	7	Halt	Terminate the program.
1000	8	Skipcond	Skip the next instruction on condition.
1001	9	Jump X	Load the value of X into PC.

MC2 Write the following code segment in MARIE assembly language:

Note:

```
int X = 1;  
while (X < 10)  
    X = X + 1;
```

<=>

```
int X = 1;  
Loop: if (X < 10) {  
    X = X + 1;  
    goto Loop;  
}
```

Instruction Number		Instruction	Meaning
Bin	Hex		
0001	1	Load X	Load the contents of address X into AC.
0010	2	Store X	Store the contents of AC at address X.
0011	3	Add X	Add the contents of address X to AC and store the result in AC.
0100	4	Subt X	Subtract the contents of address X from AC and store the result in AC.
0101	5	Input	Input a value from the keyboard into AC.
0110	6	Output	Output the value in AC to the display.
0111	7	Halt	Terminate the program.
1000	8	Skipcond	Skip the next instruction on condition.
1001	9	Jump X	Load the value of X into PC.

MC2 Write the following code segment in MARIE assembly language:

Solution:

```
int X = 1;  
while (X < 10)  
    X = X + 1;
```

<=>

```
int X = 1;  
Loop: if (X < 10) {  
    X = X + 1;  
    goto Loop;  
}
```

```
Loop,      Load      X          / Load loop constant  
           Subt      Ten        / Compare X to 10  
           SkipCond  000        / If AC<0 (X is less than 10), continue loop  
           Jump      Endloop / If X is not less than 10, terminate loop  
  
           Load      X          / Begin body of loop  
           Add       One       / Add 1 to X  
           Store     X          / Store new value in X  
  
           Jump      Loop      / Continue loop  
  
EndLoop,   Halt                / Terminate program  
  
X,         Dec       1          / Storage for X (value 1 given in problem)  
One,       Dec       1          / The constant value 1  
Ten,      Dec       10         / The loop constant
```

MC3 Write the following code segment in MARIE assembly language:

```
int X = 1;  
do {  
    X = X + 1;  
} while (X < 10)
```

Instruction Number		Instruction	Meaning
Bin	Hex		
0001	1	Load X	Load the contents of address X into AC.
0010	2	Store X	Store the contents of AC at address X.
0011	3	Add X	Add the contents of address X to AC and store the result in AC.
0100	4	Subt X	Subtract the contents of address X from AC and store the result in AC.
0101	5	Input	Input a value from the keyboard into AC.
0110	6	Output	Output the value in AC to the display.
0111	7	Halt	Terminate the program.
1000	8	Skipcond	Skip the next instruction on condition.
1001	9	Jump X	Load the value of X into PC.

MC3 Write the following code segment in MARIE assembly language:

Note:

```
int X = 1;  
do {  
    X = X + 1;  
} while (X < 10)
```

<=>

```
int X = 1;  
Loop: X = X + 1;  
    if (X < 10)  
        goto Loop;
```

Instruction Number		Instruction	Meaning
Bin	Hex		
0001	1	Load X	Load the contents of address X into AC.
0010	2	Store X	Store the contents of AC at address X.
0011	3	Add X	Add the contents of address X to AC and store the result in AC.
0100	4	Subt X	Subtract the contents of address X from AC and store the result in AC.
0101	5	Input	Input a value from the keyboard into AC.
0110	6	Output	Output the value in AC to the display.
0111	7	Halt	Terminate the program.
1000	8	Skipcond	Skip the next instruction on condition.
1001	9	Jump X	Load the value of X into PC.

MC3 Write the following code segment in MARIE assembly language:

Solution:

```
int X = 1;
do {
    X = X + 1;
} while (X < 10)
```

<=>

```
int X = 1;
Loop: X = X + 1;
      if (X < 10)
          goto Loop;
```

```
Loop,    Load    X        / Begin body of loop
         Add      One      / Add 1 to X
         Store    X        / Store new value in X

         Load     X        / Load loop constant
         Subt     Ten      / Compare X to 10
         SkipCond 000      / If AC<0 (X is less than 10), continue loop
         Jump      Endloop / If X is not less than 10, terminate loop
         Jump      Loop   / Continue loop

EndLoop, Halt                    / Terminate program

X,       Dec      1          / Storage for X (value 1 given in problem)
One,     Dec      1          / The constant value 1
Ten,     Dec      10         / The loop constant
```

MC4 Write the following code segment in MARIE assembly language:

```
int X = 1, Sum = 0;
while (X < 11) {
    Sum = Sum + X;
    X = X + 1;
}
printf("%d", Sum);
```

Instruction Number		Instruction	Meaning
Bin	Hex		
0001	1	Load X	Load the contents of address X into AC.
0010	2	Store X	Store the contents of AC at address X.
0011	3	Add X	Add the contents of address X to AC and store the result in AC.
0100	4	Subt X	Subtract the contents of address X from AC and store the result in AC.
0101	5	Input	Input a value from the keyboard into AC.
0110	6	Output	Output the value in AC to the display.
0111	7	Halt	Terminate the program.
1000	8	Skipcond	Skip the next instruction on condition.
1001	9	Jump X	Load the value of X into PC.

MC4 Write the following code segment in MARIE assembly language:

Solution:

```
Loop,      Load      X          / AC ← X
           Subt       Eleven     / AC ← X-11
           SkipCond   000        / Test if AC<0 (X < 11)
           Jump       Endloop    / False; Terminate loop
```

```
           Load      Sum
           Add        X          / Add X to Sum
           Store      Sum        / Store result in Sum
```

```
           Load      X
           Add        One        / Increment X
           Store      X
```

```
           Jump       Loop
```

```
Endloop,   Load      Sum
           Output     / Print Sum
           Halt       / Terminate program
```

```
Sum,       Dec      0          / Storage for Sum (value 0 given in problem)
X,         Dec      1          / Storage for X (value 1 given in problem)
One,       Dec      1          / The constant 1
Eleven,    Dec      11         / The constant 11
```

```
int X = 1, Sum = 0;
Loop: if (X < 11) {
    Sum = Sum + X;
    X = X + 1;
    goto Loop;
}
printf("%d", Sum);
```


MC5 Solve again exercise MC4, but this time with $X \leq 11$, instead of $X < 11$.

```
int X = 1, Sum = 0;
while (X <= 11) {
    Sum = Sum + X;
    X = X + 1;
}
printf("%d", Sum);
```

Instruction Number		Instruction	Meaning
Bin	Hex		
0001	1	Load X	Load the contents of address X into AC.
0010	2	Store X	Store the contents of AC at address X.
0011	3	Add X	Add the contents of address X to AC and store the result in AC.
0100	4	Subt X	Subtract the contents of address X from AC and store the result in AC.
0101	5	Input	Input a value from the keyboard into AC.
0110	6	Output	Output the value in AC to the display.
0111	7	Halt	Terminate the program.
1000	8	Skipcond	Skip the next instruction on condition.
1001	9	Jump X	Load the value of X into PC.

MC5 Solve again exercise MC4, but this time with $X \leq 11$, instead of $X < 11$.

Solution a): test if $X < 11$ and, if that is false, test if $X == 11$

```
Loop,      Load      X          / AC ← X
           Subt       Eleven     / AC ← X-11
If,      SkipCond 000         / Test if AC<0 (X < 11)
           Jump ElseIf         / False; Jump to "Test if AC==0"
           Jump Then          / True; Jump to loop body
ElseIf,  SkipCond 400         / Test if AC==0 (X == 11)
           Jump       Endloop    / False; Terminate loop

Then,    Load      Sum
           Add        X          / Add X to Sum
           Store      Sum        / Store result in Sum

           Load      X
           Add        One        / Increment X
           Store      X

           Jump       Loop

Endloop,   Load      Sum
           Output
           Halt        / Terminate program
...

```

```
int X = 1, Sum = 0;
Loop: if (X <= 11) {
    Sum = Sum + X;
    X = X + 1;
    goto Loop;
}
printf("%d", Sum);

```

MC5 Solve again exercise MC4, but this time with $X \leq 11$, instead of $X < 11$.

Solution b): test if $X < 11+1$

```
Loop,      Load      X          / AC ← X
           Subt       Eleven     / AC ← X-11
           Subt       One       / AC ← (X-11)-1 <=> AC ← X-12
           SkipCond  000        / Test if AC<0 (X < 12)
           Jump      Endloop    / False; Terminate loop

           Load      Sum
           Add        X          / Add X to Sum
           Store      Sum        / Store result in Sum

           Load      X
           Add        One        / Increment X
           Store      X

           Jump      Loop

Endloop,   Load      Sum
           Output
           Halt
...
           / Print Sum
           / Terminate program
```

```
int X = 1, Sum = 0;
Loop: if (X < 12) {
    Sum = Sum + X;
    X = X + 1;
    goto Loop;
}
printf("%d", Sum);
```

MC5 Solve again exercise MC4, but this time with $X \leq 11$, instead of $X < 11$.

Solution c): test if $X > 11$

```
Loop,      Load      X          / AC ← X
           Subt       Eleven    / AC ← X-11
If,      SkipCond 800        / Test if AC>0 (X > 11)
           Jump Then          / False; Jump to loop body
           Jump       Endloop    / True; Terminate loop

Then,     Load      Sum
           Add        X          / Add X to Sum
           Store      Sum        / Store result in Sum

           Load      X
           Add        One        / Increment X
           Store      X

           Jump       Loop

Endloop,   Load      Sum
           Output                     / Print Sum
           Halt                    / Terminate program
...

```

```
int X = 1, Sum = 0;
Loop: if (X > 11)
    goto EndLoop;
else {
    Sum = Sum + X;
    X = X + 1;
    goto Loop;
}
EndLoop: printf("%d", Sum);

```

MC6 Write the following code segment in MARIE assembly language:

```
int i, a[5]={'0','2','4','6','8'};

for(i=0; i<5; i++)
    printf("%c", a[i]);
```

Instruction Number		Instruction	Meaning
Bin	Hex		
0001	1	Load X	Load the contents of address X into AC.
0010	2	Store X	Store the contents of AC at address X.
0011	3	Add X	Add the contents of address X to AC and store the result in AC.
0100	4	Subt X	Subtract the contents of address X from AC and store the result in AC.
0101	5	Input	Input a value from the keyboard into AC.
0110	6	Output	Output the value in AC to the display.
0111	7	Halt	Terminate the program.
1000	8	Skipcond	Skip the next instruction on condition.
1001	9	Jump X	Load the value of X into PC.

Instruction Number (hex)	Instruction	Meaning
0	JnS X	Store the PC at address X and jump to X + 1.
A	Clear	Put all zeros in AC.
B	AddI X	Add indirect: Go to address X. Use the value at X as the actual address of the data operand to add to AC.
C	JumpI X	Jump indirect: Go to address X. Use the value at X as the actual address of the location to jump to.
D	LoadI X	Load indirect: Go to address X. Use the value at X as the actual address of the operand to load into the AC.
E	StoreI X	Store indirect: Go to address X. Use the value at X as the destination address for storing the value in AC.

MC6 Write the following code segment in MARIE assembly language:

Note:

```
int i, a[5]={'0','2','4','6','8'};
for(i=0; i<5; i++)
    printf("%c", a[i]);
```

<=>

```
int i=0;
int a[5]={'0','2','4','6','8'};
Loop: if (i < 5) {
    printf("%c", a[i]);
    i = i + 1;
    goto Loop;
}
```

Instruction Number		Instruction	Meaning
Bin	Hex		
0001	1	Load X	Load the contents of address X into AC.
0010	2	Store X	Store the contents of AC at address X.
0011	3	Add X	Add the contents of address X to AC and store the result in AC.
0100	4	Subt X	Subtract the contents of address X from AC and store the result in AC.
0101	5	Input	Input a value from the keyboard into AC.
0110	6	Output	Output the value in AC to the display.
0111	7	Halt	Terminate the program.
1000	8	Skipcond	Skip the next instruction on condition.
1001	9	Jump X	Load the value of X into PC.

Instruction Number (hex)	Instruction	Meaning
0	JnS X	Store the PC at address X and jump to X + 1.
A	Clear	Put all zeros in AC.
B	AddI X	Add indirect: Go to address X. Use the value at X as the actual address of the data operand to add to AC.
C	JumpI X	Jump indirect: Go to address X. Use the value at X as the actual address of the location to jump to.
D	LoadI X	Load indirect: Go to address X. Use the value at X as the actual address of the operand to load into the AC.
E	StoreI X	Store indirect: Go to address X. Use the value at X as the destination address for storing the value in AC.

MC6 Write the following code segment in MARIE assembly language:

Solution:

```
ORG 100

Loop,   Load      ArrayCellIndex
        Subt       Five
        Skipcond   000          / check if i < 5
        Jump       EndLoop

        Load       ArrayBaseAddress
        Add         ArrayCellIndex
        Store       ArrayCellAddress / new address of a[i]

        LoadI      ArrayCellAddress / load a[i]
        Output      / output a[i]

        Load       ArrayCellIndex
        Add         One
        Store       ArrayCellIndex / i++

        Jump       Loop

EndLoop, Halt
```

```
int i=0;
int a[5]={'0','2','4','6','8'};
Loop: if (i < 5) {
        printf("%c", a[i]);
        i = i + 1;
        goto Loop;
}
```

	Dec 48	/ a[0]
	Dec 50	/ a[1]
	Dec 52	/ a[2]
	Dec 54	/ a[3]
	Dec 56	/ a[4]
ArrayBaseAddress,	Hex 10E	/ address of a[0]
ArrayCellAddress,	Hex 000	/ address of a[i]
ArrayCellIndex,	Dec 0	/ index i
One,	Dec 1	/ constant 1
Five,	Dec 5	/ constant 5

MC7 What are the potential problems with the following assembly code fragment (implementing a subroutine) written to run on MARIE? The subroutine assumes its parameter is in AC and doubles its value. The Main part of the program includes a sample call to the subroutine. You can assume this fragment is part of a larger program.

```

Main,      Load X
           Jump Sub1

Sret,      Store X
...

Sub1,      Add X
           Jump  Sret
```


MC7 What are the potential problems with the following assembly code fragment (implementing a subroutine) written to run on MARIE? The subroutine assumes its parameter is in AC and doubles its value. The Main part of the program includes a sample call to the subroutine. You can assume this fragment is part of a larger program.

Solution:

First Problem: this subroutine works only for the parameter X (no other variable could be used as X is explicitly added in the subroutine)

Second Problem: this subroutine cannot be called from anywhere, as it always returns to Sret

```

Main,      Load X
           Jump Sub1
Sret,      Store X
...
Sub1,      Add X
           Jump Sret
```

MC8 Write a MARIE subroutine to subtract two numbers and use the subroutine to help calculate $E=A-B$ and $F=C-D$.

Subroutine calling (in C): execution sequence

```
int main()  
{  
    int A, B, E;  
    scanf("%d%d", &A, &B);  
    E = subr(A, B);  
    printf("%d", E);  
    return 0;  
}
```

x and **y** are the **formal** parameters of the function **subr** (i.e., they are place-holders in the signature / declaration of the function).

```
int subr ( int x, int y )  
{  
    int res;  
  
    res = x - y;  
    return res;  
}
```

A and **B** are the **actual** parameters of the function **subr** (i.e., they are the values that are actually passed to the function)

MC8 Write a MARIE subroutine to subtract two numbers and use the subroutine to help calculate $E=A-B$ and $F=C-D$.

Subroutine calling (in C): parameter passing

```
int main()
{
    int A, B, E;
    scanf("%d%d", &A, &B);

    E = subr(A, B);

    printf("%d", E);
    return 0;
}
```

```
int subr(int x, int y)
{
    int res;

    res = x - y;
    return res;
}
```

The value of **A** is copied to the variable **x** of function **subr**.

The value of **B** is copied to the variable **y** of function **subr**.

MC8 Write a MARIE subroutine to subtract two numbers and use the subroutine to help calculate $E = A - B$ and $F = C - D$.

Subroutine calling (in C): result returning

```
int main()
{
    int A, B, E;
    scanf("%d%d", &A, &B);

    E ← subr(A, B);

    printf("%d", E);
    return 0;
}
```

```
int subr(int x, int y)
{
    int res;

    res = x - y;
    return res;
}
```

The value of **res** is copied to the variable **E** of function **main**.

MC8 Write a MARIE subroutine to subtract two numbers and use the subroutine to help calculate $E=A-B$ and $F=C-D$.

Complete C version:

```
int Subr (int X, int Y)
{
    int Res;
    Res = X - Y;
    return (Res);
}
Main()
{
    int A=8,B=4,C=10,D=2,E,F;
    E=Subr(A,B);printf("%d",E);
    F=Subr(C,D);printf("%d",F);
}
```

MC8 Write a MARIE subroutine to subtract two numbers and use the subroutine to help calculate $E=A-B$ and $F=C-D$.

Solution:

```
Load   A      / Load the first number (A)
Store  X      / Use X as a parameter to pass A to Subr
Load   B      / Load the second number (B)
Store  Y      / Use Y as a parameter to pass B to Subr
JnS    Subr   / Store return address and jump to Subr
Load   Res    / Load the result returned by Subr
Store  E      / Store the result
Output / Output the result
```

```
Load   C      / Load the first number (C)
Store  X      / Use X as a parameter to pass C to Subr
Load   D      / Load the second number (D)
Store  Y      / Use Y as a parameter to pass D to Subr
JnS    Subr   / Store return address and jump to Subr
Load   Res    / Load the result returned by Subr
Store  F      / Store the result
Output / Output the result
```

```
Halt      / End program
```

```
Subr, Hex 0    / Place to store return address
Load  X     / Load the first number
Subt  Y     / Subtract the second number
Store Res   / Store result
JumpI Subr  / Return to calling code
```

```
int Subr (int X, int Y)
{
    int Res;
    Res = X - Y;
    return(Res);
}
Main()
{
    int A=8,B=4,C=10,D=2,E,F;
    E=Subr(A,B);printf("%d",E);
    F=Subr(C,D);printf("%d",F);
}
```

A,	Dec	8	/ Arbitrary value
B,	Dec	4	/ Arbitrary value
C,	Dec	10	/ Arbitrary value
D,	Dec	2	/ Arbitrary value
E,	Dec	0	
F,	Dec	0	
X,	Dec	0	/ Subroutine parameter
Y,	Dec	0	/ Subroutine parameter
Res,	Dec	0	/ Subroutine result

MC9 Write a MARIE program using a loop that multiplies two positive numbers by using repeated addition. For example, to multiple 3 x 6, the program would add 3 six times, or $3+3+3+3+3+3$.

Hint: start by writing the program in C and then translate this program to MARIE assembly.

MC9 Write a MARIE program using a loop that multiplies two positive numbers by using repeated addition. For example, to multiple 3 x 6, the program would add 3 six times, or 3+3+3+3+3+3.

C version:

```
int x=3, y=6, sum, ctr;
sum=0; ctr=y;
while (ctr > 0) {
    sum = sum + x;
    ctr--;
}
printf("%d", sum);
```

NOTE: this version assumes that $y \geq 0$

MC9 Write a MARIE program using a loop that multiplies two positive numbers by using repeated addition. For example, to multiple 3 x 6, the program would add 3 six times, or 3+3+3+3+3+3.

Solution:

```

Clear
Store    Sum    / Init Sum ( Sum = 0 )
Load     Y
Store    Ctr    / Init Ctr ( Ctr = Y )

Loop,    Load    Ctr
SkipCond 800    / Ctr > 0 ?
Jump     EndLoop / False: break the loop
Load     Sum    / True: keep on the loop
Add      X
Store    Sum    / Update Sum ( Sum = Sum + X )
Load     Ctr
Subt     One
Store    Ctr    / Update Ctr ( Ctr -- )
Jump     Loop   / Repeat the loop

EndLoop, Load    Sum
Output   / Print Sum ( product of X by Y )
Halt     / End program

X,       Dec     3    / Initial value of X
Y,       Dec     6    / Initial value of Y
Sum,     Dec     0    / Result of multiplication
Ctr,     Dec     0    / Counter for looping
One,     Dec     1    / The constant value 1
```

```
int x=3, y=6, sum, ctr;
sum=0; ctr=y;
while (ctr > 0) {
    sum = sum + x;
    ctr--;
}
printf("%d", sum);
```

<=>

```
int x=3, y=6, sum, ctr;
sum=0; ctr=y;
Loop: if (ctr > 0) {
    sum = sum + x;
    ctr--;
    goto Loop;
}
printf("%d", sum);
```

MC10 Transform the solution of exercise MC9 into a multiplication subroutine Mul and use it to evaluate the following expression: $G = (E = A \times B) + (F = C \times D)$.

MC10 Transform the solution of exercise MC9 into a multiplication subroutine Mul and use it to evaluate the following expression: $G = (E = A \times B) + (F = C \times D)$.

Solution – Subroutine Mul:

```
Mul,      Hex      0    / Place to store return address
          Clear
          Store     Sum  / Init Sum ( Sum = 0 )
          Load      Y
          Store     Ctr  / Init Ctr ( Ctr = Y )

Loop,     Load      Ctr
          SkipCond 800  / Ctr > 0 ?
          Jump      EndLoop / False: break the loop
          Load      Sum  / True: keep on the loop
          Add        X
          Store     Sum  / Update Sum ( Sum = Sum + X )
          Load      Ctr
          Subt       One
          Store     Ctr  / Update Ctr ( Ctr -- )
          Jump      Loop  / Repeat the loop

EndLoop,  JumpI      Mul    / Done with subroutine, return to main

X,        Dec      0    / First parameter of subroutine
Y,        Dec      0    / Second parameter of subroutine
Sum,      Dec      0    / Result of subroutine
Ctr,      Dec      0    / Counter for looping
One,      Dec      1    / Constant with value 1
```

```
int Mul (int x, int y)
{
    int sum, ctr;
    sum=0; ctr=y;
    while (ctr > 0) {
        sum = sum + x;
        ctr--;
    }
    return(sum);
}
```

MC10 Transform the solution of exercise MC9 into a multiplication subroutine Mul and use it to evaluate the following expression: $G = (E = A \times B) + (F = C \times D)$.

Solution – Main Program:

```
Load  A
Store X    / Store A in first parameter
Load  B
Store Y    / Store B in second parameter
JnS    Mul / Jump to multiplication subroutine
Load  Sum / Get result
Store E    / E = A x B

Load  C
Store X    / Store C in first parameter
Load  D
Store Y    / Store D in second parameter
JnS    Mul / Jump to multiplication subroutine
Load  Sum / Get result
Store F    / F = C x D

Load  E    / Get first result
Add   F    / Add second result
Store G    / G = E + F
Output    / Show final result
Halt      / Terminate program
```

```
main() {
    int A,B,C,D,E,F,G;

    E=Mul(A,B);
    F=Mul(C,D);
    G=E+F;

    printf("%d", G);
}
```

A,	Dec	1	/ Arbitrary value
B,	Dec	2	/ Arbitrary value
C,	Dec	3	/ Arbitrary value
D,	Dec	4	/ Arbitrary value
E,	Dec	0	/ Storage for first result
F,	Dec	0	/ Storage for second result
G,	Dec	0	/ Storage for final result

MC11 Write the following subroutines in MARIE assembly (and small programs that use them):

- a) A subroutine “Min” to get the minimum of two integers.
- b) A subroutine “Max” to get the maximum of two integers.
- c) A subroutine “Sym” to get the symmetric of one integer.
- d) A subroutine “Div” to get the integer quotient of the division of two integers.
- e) A subroutine “Mod” to get the integer remainder of the division of two integers.
- f) A subroutine “Prime” to check if an integer is prime.
- g) A subroutine “Mul2” that optimizes Mul, by minimizing the n° of successive additions using the Min and Max subroutines.

Note: in the MARIE code, to avoid name collisions (tags and variables with the same name, in different subroutines), the name of function is used as a prefix (e.g., Func_X is the tag/variable X of function Func).

MC11 Write the following subroutines in MARIE assembly (and small programs that use them):

a) A subroutine “Min” to get the minimum of two integers.

Solution – Main Program:

C:

```
main()  
{  
    int x, y, z;  
    scanf("%d%d",&x,&y);  
    z=Min(x,y);  
    printf("%d",z);  
}
```

MARIE:

Main,	Input	
	Store	Main_X
	Input	
	Store	Main_Y
	Load	Main_X
	Store	Min_X
	Load	Main_Y
	Store	Min_Y
	JnS	Min
	Load	Min_Z
	Store	Main_Z
	Load	Main_Z
	Output	
	Halt	
Main_X,	Dec	0
Main_Y,	Dec	0
Main_Z,	Dec	0

MC11 Write the following subroutines in MARIE assembly (and small programs that use them):

a) A subroutine “Min” to get the minimum of two integers.

Solution – Subroutine Min:

C:

```
int Min (int x, int y)
{
    int z;
    if (x < y)
        z = x;
    else
        z = y;
    return(z);
}
```

MARIE:

```
Min,          Hex 0

Min_If,        Load    Min_X
               Subt     Min_Y
               Skipcond 000
               Jump     Min_Else

Min_Then,      Load    Min_X
               Store    Min_Z
               Jump     Min_EndIf

Min_Else,      Load    Min_Y
               Store    Min_Z

Min_EndIf,     JumpI    Min

Min_X,         Dec 0
Min_Y,         Dec 0
Min_Z,         Dec 0
```

MC11 Write the following subroutines in MARIE assembly (and small programs that use them):

b) A subroutine “Max” to get the maximum of two integers.

Solution – Main Program:

C:

```
main()
{
    int x, y, z;
    scanf("%d%d", &x, &y);
    z=Max(x,y);
    printf("%d", z);
}
```

MARIE:

Main,	Input	
	Store	Main_X
	Input	
	Store	Main_Y
	Load	Main_X
	Store	Max_X
	Load	Main_Y
	Store	Max_Y
	JnS	Max
	Load	Max_Z
	Store	Main_Z
	Load	Main_Z
	Output	
	Halt	
Main_X,	Dec	0
Main_Y,	Dec	0
Main_Z,	Dec	0

MC11 Write the following subroutines in MARIE assembly (and small programs that use them):

b) A subroutine “Max” to get the maximum of two integers.

Solution – Subroutine Max:

C:

```
int Max (int x, int y)
{
    int z;
    if (x < y)
        z = y;
    else
        z = x;
    return(z);
}
```

MARIE:

```
Max,      Hex 0

Max_If,    Load    Max_X
           Subt     Max_Y
           Skipcond 000
           Jump     Max_Else

Max_Then,  Load     Max_Y
           Store     Max_Z
           Jump     Max_EndIf

Max_Else,  Load     Max_X
           Store     Max_Z

Max_EndIf, JumpI     Max

Max_X,     Dec 0
Max_Y,     Dec 0
Max_Z,     Dec 0
```

MC11 Write the following subroutines in MARIE assembly (and small programs that use them):

c) A subroutine “Sym” to get the symmetric of one integer.

Solution (C):

```
int Sym (int x)
{
    int y;
    Y = 0 - x;
    return(y) ;
}

main()
{
    int x, y;
    scanf ("%d" , &x) ;
    y=Sym(x) ;
    printf ("%d" , y) ;
}
```

Solution
(MARIE):

```
Main,      Input
            Store   Main_X

            Load    Main_X
            Store    Sym_X
            JnS      Sym
            Load     Sym_Y
            Store     Main_Y

            Load     Main_Y
            Output

            Halt

Main_X,     Dec 0
Main_Y,     Dec 0

//////////
Sym,        Hex 0
            Clear
            Subt     Sym_X
            Store    Sym_Y
            JumpI    Sym

Sym_X,      Dec 0
Sym_Y,      Dec 0
```

MC11 Write the following subroutines in MARIE assembly (and small programs that use them):

d) A subroutine “Div” to get the integer quotient of the division of two integers.

Solution (MARIE):

Solution (C):

```
int Div (int x, int y)
{
    int z=x, ctr=0;
    // while (z >= y) {
    while (z > y - 1) {
        z = z - y;
        ctr = ctr + 1;
    }
    return(ctr);
}

main()
{
    int x, y, z;
    scanf("%d%d", &x, &y);
    z=Div(x,y);
    printf("%d", z);
}
```

```
Div,          Hex 0
Load          Div_X
Store         Div_Z
Clear
Store         Div_Ctr
Div_Loop,     Load      Div_Z
              Subt      Div_Y
              Add       One
              Skipcond 800
              //Jump     Div_EndLoop
              JumpI     Div
              Load      Div_Z
              Subt      Div_Y
              Store     Div_Z
              Load      Div_Ctr
              Add       One
              Store     Div_Ctr
              Jump      Div_Loop
              //Div_EndLoop, JumpI Div
Div_X,        Dec      0
Div_Y,        Dec      0
Div_Z,        Dec      0
Div_Ctr,      Dec      0
One,          Dec      1
// Note: Main similar to MC11.a)
```

MC11 Write the following subroutines in MARIE assembly (and small programs that use them):

d.opt1) A subroutine “Div” to get the integer quotient of the division of two integers.

Solution (MARIE):

Solution (C):

```
int Div (int x, int y)
{
    int z=x, ctr=0;
    BeginLoop: if (z < y) goto EndLoop;
    BodyLoop:  z = z - y;
               ctr = ctr + 1;
               goto Loop;
    EndLoop:   return(ctr);
}

main()
{
    int x, y, z;
    scanf("%d%d", &x, &y);
    z=Div(x,y);
    printf("%d", z);
}
```

```
Div,          Hex 0
              Load    Div_X
              Store    Div_Z
              Clear
              Store    Div_Ctr
Div_BeginLoop, Load    Div_Z
              Subt     Div_Y
              Skipcond 000
              Jump     Div_BodyLoop
              //      Jump     Div_EndLoop
              //      JumpI    Div
              // Div_BodyLoop, Load    Div_Z
              //      Subt     Div_Y
Div_BodyLoop, Store    Div_Z
              Load    Div_Ctr
              Add      One
              Store    Div_Ctr
              Jump     Div_BeginLoop
              //Div_EndLoop, JumpI    Div
Div_X,        Dec      0
Div_Y,        Dec      0
Div_Z,        Dec      0
Div_Ctr,      Dec      0
One,          Dec      1
              // Note: Main similar to MC11.a)
```

MC11 Write the following subroutines in MARIE assembly (and small programs that use them):

e) A subroutine “Mod” to get the integer remainder of the division of two integers.

Solution (C):

```
int Mod (int x, int y)
{
    int z=x;
    //while (z >= y) {
    while (z > y - 1) {
        z = z - y;
    }
    return(z);
}

main()
{
    int x, y, z;
    scanf("%d%d", &x, &y);
    z=Mod(x,y);
    printf("%d", z);
}
```

Solution (MARIE):

```
Mod,          Hex 0
              Load    Mod_X
              Store    Mod_Z
Mod_Loop,     Load    Mod_Z
              Subt     Mod_Y
              Add      One
              Skipcond 800
              //      Jump    Mod_EndLoop
              JumpI    Mod
              Load     Mod_Z
              Subt     Mod_Y
              Store     Mod_Z
              Jump     Mod_Loop
              //Mod_EndLoop, JumpI    Mod
Mod_X,        Dec     0
Mod_Y,        Dec     0
Mod_Z,        Dec     0
One,          Dec     1

// Note: Main similar to MC11.a)
```

MC11 Write the following subroutines in MARIE assembly (and small programs that use them):

e.opt1) A subroutine “Mod” to get the integer remainder of the division of two integers.

Solution (C):

```
int Mod (int x, int y)
{
    int z=x;
    BeginLoop: if (z < y) goto EndLoop;
    BodyLoop:  z = z - y;
    EndLoop:   return(z);
}

main()
{
    int x, y, z;
    scanf("%d%d", &x, &y);
    z=Mod(x,y);
    printf("%d", z);
}
```

Solution (MARIE):

Mod,	Hex 0
	Load Mod_X
	Store Mod_Z
Mod_BeginLoop,	Load Mod_Z
	Subt Mod_Y
	Skipcond 000
	Jump Mod_BodyLoop
//	Jump Mod_EndLoop
	JumpI Mod
//Mod_BodyLoop,	Load Mod_Z
//	Subt Mod_Y
Mod_BodyLoop,	Store Mod_Z
	Jump Mod_BeginLoop
//Mod_EndLoop,	JumpI Mod
Mod_X,	Dec 0
Mod_Y,	Dec 0
Mod_Z,	Dec 0
One,	Dec 1

// Note: Main similar to MC11.a)

MC11 Write the following subroutines in MARIE assembly (and small programs that use them):

f) A subroutine “Prime” to check if an integer is prime.

Solution (C):

```
int Prime (int n)
{
    int p=1, i;
    if (n==0 || n==1) p=0; // n < 2
    else
        for (i=2; i < n; i++) {
            if (Mod(n,i)==0) {
                p=0; break;
            }
        }
    return(p) ;
}

main()
{
    int n, p;
    scanf ("%d", &n) ;
    p=Prime (n) ;
    printf ("%d", p) ;
}
```

MC11 Write the following subroutines in MARIE assembly (and small programs that use them):

f) A subroutine “Prime” to check if an integer is prime.

Solution (MARIE):

```
Prime,           Hex 0

                Load    One
                Store    Prime_P

Prime_If,         Load    Prime_N
                Subt     Two
                Skipcond 000
                Jump     Prime_Else

Prime_Then,       Clear
                Store    Prime_P
                Jump     Prime_EndLoop

Prime_Else,       Load    Two
                Store    Prime_I

Prime_Loop,       Load    Prime_I
                Subt     Prime_N
                Skipcond 000
                Jump     Prime_EndLoop
```

```
                Load     Prime_N
                Store     Mod_X
                Load     Prime_I
                Store     Mod_Y
                Jns       Mod
                Load     Mod_Z
                Skipcond 400
                Jump     Prime_IncCounter
                Clear
                Store     Prime_P
                Jump     Prime_EndLoop

Prime_IncCounter, Load    Prime_I
                Add      One
                Store     Prime_I
                Jump     Prime_Loop

Prime_EndLoop,   JumpI    Prime

Prime_N,         Dec     0
Prime_P,         Dec     0
Prime_I,         Dec     0
One,             Dec     1
Two,             Dec     2
```

// Note: Main similar to MC11.c)

MC11 Write the following subroutines in MARIE assembly (and small programs that use them):

f.opt1) A subroutine “Prime” to check if an integer is prime.

Solution (C):

```
int Prime (int n)
{
    int p=1, i;
    if (n==0 || n==1) p=0; // n < 2
    else
        for (i=2; Mul(i,2) - 1 - n < 0; i++)
            if (Mod(n,i)==0) {
                p=0; break;
            }
    return(p) ;
}

main()
{
    int n, p;
    scanf ("%d", &n) ;
    p=Prime (n) ;
    printf ("%d", p) ;
}
```

```
i <= n / 2
<==>
i * 2 <= n
<==>
i * 2 < n + 1
<==>
i * 2 - n - 1 < 0
<==>
Mul(i,2) - n - 1 < 0
```

MC11 Write the following subroutines in MARIE assembly (and small programs that use them):

f.opt1) A subroutine “Prime” to check if an integer is prime.

Solution (MARIE):

```

Prime,          Hex 0

                Load    One
                Store    Prime_P

Prime_If,        Load    Prime_N
                Subt     Two
                Skipcond 000
                Jump     Prime_Else

Prime_Then,      Clear
                Store    Prime_P
                Jump     Prime_EndLoop

Prime_Else,      Load    Two
                Store    Prime_I

Prime_Loop,      Load    Prime_I
                Store    Mul_X
                Load    Two
                Store    Mul_Y
                Jns      Mul
                Load    Mul_Sum
                Subt     One
                Subt     Prime_N
                Skipcond 000
    
```

```

                Jump     Prime_EndLoop

                Load    Prime_N
                Store    Mod_X
                Load    Prime_I
                Store    Mod_Y
                Jns      Mod
                Load    Mod_Z
                Skipcond 400
                Jump     Prime_IncCounter
                Clear
                Store    Prime_P
                Jump     Prime_EndLoop

Prime_IncCounter, Load    Prime_I
                Add      One
                Store    Prime_I
                Jump     Prime_Loop

Prime_EndLoop,   JumpI    Prime

Prime_N,         Dec      0
Prime_P,         Dec      0
Prime_I,         Dec      0
One,             Dec      1
Two,             Dec      2

// Note: Main similar to MC11.c)
    
```

MC11 Write the following subroutines in MARIE assembly (and small programs that use them):

f.opt2) A subroutine “Prime” to check if an integer is prime.

Solution (C):

```
int Prime (int n)
{
    int p=1, i;
    if (n==0 || n==1) p=0; // n < 2
    else
        for (i=2; Div(n,2) - i + 1 > 0; i++)
            if (Mod(n,i)==0) {
                p=0; break;
            }
    return(p) ;
}

main()
{
    int n, p;
    scanf ("%d", &n) ;
    p=Prime (n) ;
    printf ("%d", p) ;
}
```

```
    i <= n / 2
    <==>
    i < n / 2 + 1
    <==>
    i - n / 2 - 1 < 0
    <==>
    i - Div(n,2) - 1 < 0
    <==>
    -i + Div(n,2) + 1 > 0
    <==>
    Div(n,2) - i + 1 > 0
```

MC11 Write the following subroutines in MARIE assembly (and small programs that use them):

f.opt2) A subroutine “Prime” to check if an integer is prime.

Solution (MARIE):

```
Prime,           Hex 0

                Load    One
                Store    Prime_P

Prime_If,         Load    Prime_N
                Subt     Two
                Skipcond 000
                Jump     Prime_Else

Prime_Then,       Clear
                Store    Prime_P
                Jump     Prime_EndLoop

Prime_Else,       Load    Two
                Store    Prime_I

Prime_Loop,       Load    Prime_N
                Store    Div_X
                Load    Two
                Store    Div_Y
                Jns      Div
                Load    Div_Ctr
                Subt     Prime_I
                Add      One
                Skipcond 800
```

```
                Jump     Prime_EndLoop

                Load     Prime_N
                Store     Mod_X
                Load     Prime_I
                Store     Mod_Y
                Jns       Mod
                Load     Mod_Z
                Skipcond 400
                Jump     Prime_IncCounter
                Clear
                Store     Prime_P
                Jump     Prime_EndLoop

Prime_IncCounter, Load    Prime_I
                Add      One
                Store     Prime_I
                Jump     Prime_Loop

Prime_EndLoop,   JumpI    Prime

Prime_N,         Dec      0
Prime_P,         Dec      0
Prime_I,         Dec      0
One,             Dec      1
Two,             Dec      2

// Note: Main similar to MC11.c)
```

MC11 Write the following subroutines in MARIE assembly (and small programs that use them):

g) A subroutine “Mul2” that optimizes Mul, by minimizing the n° of successive additions using the Min and Max subroutines.

Solution (C):

```
int Mul (int x, int y)
{
    int sum, ctr;
    sum=0; ctr=y;
    while (ctr > 0) {
        sum = sum + x;
        ctr--;
    }
    return(sum);
}
```



```
int Mul2 (int x, int y)
{
    int sum=0, smallest, biggest;
    smallest = Min(x,y);
    biggest = Max(x,y);
    while (smallest > 0) {
        sum = sum + biggest;
        smallest--;
    }
    return(sum);
}
```

MC11 Write the following subroutines in MARIE assembly (and small programs that use them):

g) A subroutine “Mul2” that optimizes Mul, by minimizing the n° of successive additions using the Min and Max subroutines.

Solution (MARIE):

```
Mul2,      Hex      0

          Clear
          Store      Sum

          Load       Mul2_X
          Store       Min_X
          Load       Mul2_Y
          Store       Min_Y
          JnS         Min
          Load        Min_Z
          Store        Mul2_Smallest

          Load       Mul2_X
          Store       Max_X
          Load       Mul2_Y
          Store       Max_Y
          JnS         Max
          Load        Max_Z
          Store        Mul2_Biggest
```

```
Mul2_Loop, Load      Mul2_Smallest
           SkipCond 800
           Jump      EndLoop
           Load      Mul2_Sum
           Add        Mul2_Biggest
           Store      Mul2_Sum
           Load      Mul2_Smallest
           Subt       One
           Store      Mul2_Smallest
           Jump      Mul2_Loop
```

```
EndLoop,  JumpI      Mul2
```

```
Mul2_X,      Dec      0
Mul2_Y,      Dec      0
Mul2_Sum,    Dec      0
Mul2_Smallest, Dec    0
Mul2_Biggest, Dec    0
One,         Dec      1
```

// Nota: Main similar à MC11.a)

MC12 Write the following subroutines in MARIE assembly (and small programs that use them):

- a) A subroutine “MinArray” to get the minimum of an array of integers.
- b) A subroutine “MaxArray” to get the maximum of an array of integers.
- c) A subroutine “ShowSymArray” that shows the symmetric of each element of an array.
- d) A subroutine “TestPrimeArray” that scans an array of integers and, for each integer, shows 1 (0), if the integer is (not) prime.

Note: in the C and MARIE code of these exercises, in addition to using the subroutine names as a prefix for tags and variable names, the suffixes Input and Output are also used for variables, to tell if they are used as input parameters or to return outputs.

MC12 Write the following subroutines in MARIE assembly (and small programs that use them):

a) A subroutine “MinArray” to get the minimum of an array of integers.

Solution (C):

```
int Min (int Min Input1, int Min Input2) { // see MC11.a) }  
  
int MinArray (int MinArray_Array[], int MinArray_ArrayNumCells)  
{  
    int MinArray_Output=MinArray_Array[0];  
    for (int i=1; i<MinArray_ArrayNumCells; i++)  
        MinArray_Output = Min(MinArray_Output,MinArray_Array[i]);  
    return (MinArray_Output);  
}  
main()  
{  
    int Main_Array[5]={3,2,1,4,5}, Main_ArrayNumCells=5, Main_Min;  
    Main_Min = MinArray(Main_Array, Main_ArrayNumCells);  
    printf("%d", Main_Min)  
}
```


MC12 Write the following subroutines in MARIE assembly (and small programs that use them):

a) A subroutine “MinArray” to get the minimum of an array of integers.

Solution (MARIE):

```
                                ORG 100

Main,                          Load   Main_ArrayBaseAddress
                               Store   MinArray_ArrayBaseAddress
                               Load    Main_ArrayNumCells
                               Store    MinArray_ArrayNumCells
                               JnS     MinArray
                               Load    MinArray_Output
                               Store    Main_Min
                               Output
                               Halt

Main_Array,                    Dec 3
                               Dec 2
                               Dec 1
                               Dec 4
                               Dec 5

Main_ArrayBaseAddress,        Hex 109 / address of a[0]
Main_ArrayNumCells,          Dec 5   / number of array cells
Main_Min,                    Dec 0
```

MC12

.a)

Solution (MARIE - cont) :

Min,	Hex 0	
	/ ...	
Min_EndIf,	JumpI	Min
Min_Input1,	Dec 0	
Min_Input2,	Dec 0	
Min_Output,	Dec 0	

```

MinArray,                                Hex 0

LoadI  MinArray_ArrayBaseAddress
Store  MinArray_Output                    / a[0] is the starting minimum
Load   One
Store  MinArray_ArrayCellIndex           / i <- 1

MinArray_Loop,
Load   MinArray_ArrayCellIndex
Subt   MinArray_ArrayNumCells
Skipcond 000                             / check if i < n
Jump   MinArray_EndLoop

Load   MinArray_ArrayBaseAddress
Add    MinArray_ArrayCellIndex
Store  MinArray_ArrayCellAddress         / new address of a[i]

Load   MinArray_Output
Store  Min_Input1                        / pass current minimum
LoadI  MinArray_ArrayCellAddress
Store  Min_Input2                        / pass new candidate
JnS    Min                               / assess new minimum
Load   Min_Output
Store  MinArray_Output                   / update current minimum

Load   MinArray_ArrayCellIndex
Add    One
Store  MinArray_ArrayCellIndex           / i++

Jump   MinArray_Loop

MinArray_EndLoop,  JumpI  MinArray

MinArray_ArrayBaseAddress, Hex 0 / address of a[0]; input parameter
MinArray_ArrayNumCells,   Dec 0 / number of array cells; input parameter

MinArray_ArrayCellAddress, Hex 0 / address of a[i]
MinArray_ArrayCellIndex,   Dec 0 / index i

MinArray_Output,           Dec 0
One,                       Dec 1 / constant 1

```

MC12 Write the following subroutines in MARIE assembly (and small programs that use them):

c) A subroutine “ShowSymArray” that shows the symmetric of each element of an array.

Solution (C):

```
int Sym (int Sym Input) { // see MC11.c) }

void ShowSymArray (int ShowSymArray_Array[],
                  int ShowSymArray_ArrayNumCells)
{
    for (int i=0; i<ShowSymArray_ArrayNumCells; i++)
        printf("%d",Sym(ShowSymArray_Array[i]));
}

main()
{
    int Main_Array[5]={1,2,3,4,5}, Main_ArrayNumCells=5;
    ShowSymArray(Main_Array, Main_ArrayNumCells);
}
```

MC12 Write the following subroutines in MARIE assembly (and small programs that use them):

c) A subroutine “ShowSymArray” that shows the symmetric of each element of an array.

**Solution
(MARIE) :**

```
                                ORG 100

Main,                          Load    Main_ArrayBaseAddress
                               Store    ShowSymArray_ArrayBaseAddress
                               Load     Main_ArrayNumCells
                               Store    ShowSymArray_ArrayNumCells
                               JnS      ShowSymArray
                               Halt

Main_Array,                    Dec 1
                               Dec 2
                               Dec 3
                               Dec 4
                               Dec 5

Main_ArrayBaseAddress,        Hex 106 / address of a[0]
Main_ArrayNumCells,          Dec 5   / number of array cells
```

Solution (MARIE - cont) :

```
Sym,      Hex 0
          Clear
          Subt Sym_Input
          Store Sym_Output
          JumpI Sym
```

```
Sym_Input, Dec 0
Sym_Output, Dec 0
```

```
ShowSymArray,      Hex 0

                      Clear
                      Store ShowSymArray_ArrayCellIndex / i <- 0 ; mandatory !

ShowSymArray_Loop,  Load ShowSymArray_ArrayCellIndex
                      Subt ShowSymArray_ArrayNumCells
                      Skipcond 000 / check if i < n
                      Jump ShowSymArray_EndLoop

                      Load ShowSymArray_ArrayBaseAddress
                      Add ShowSymArray_ArrayCellIndex
                      Store ShowSymArray_ArrayCellAddress / new address of a[i]

                      LoadI ShowSymArray_ArrayCellAddress / load a[i]
                      Store Sym_Input
                      JnS Sym
                      Load Sym_Output
                      Output / outputs Sym(a[i])

                      Load ShowSymArray_ArrayCellIndex
                      Add One
                      Store ShowSymArray_ArrayCellIndex / i++

                      Jump ShowSymArray_Loop

ShowSymArray_EndLoop, JumpI ShowSymArray

ShowSymArray_ArrayBaseAddress, Hex 0 / address of a[0]; input parameter
ShowSymArray_ArrayNumCells, Dec 0 / number of array cells; input parameter

ShowSymArray_ArrayCellAddress, Hex 0 / address of a[i]
ShowSymArray_ArrayCellIndex, Dec 0 / index i

One, Dec 1 / constant 1
```

MC12 Write the following subroutines in MARIE assembly (and small programs that use them):

d) A subroutine “TestPrimeArray” that scans an array of integers and, for each integer, shows 1 (0), if the integer is (not) prime.

Solution (C):

```
int Prime (int n) { // see MC11.f) }  
  
void TestPrimeArray (int TestPrimeArray_Array[],  
                     int TestPrimeArray_ArrayNumCells)  
{  
    for (int i=0; i<TestPrimeArray_ArrayNumCells; i++)  
        printf("%d",Prime(TestPrimeArray_Array[i]));  
}  
main()  
{  
    int Main_Array[5]={1,2,3,4,5}, Main_ArrayNumCells=5;  
    TestPrimeArray(Main_Array, Main_ArrayNumCells);  
}
```

MC12 Write the following subroutines in MARIE assembly (and small programs that use them):

d) A subroutine “TestPrimeArray” that scans an array of integers and, for each integer, shows 1 (0), if the integer is (not) prime.

**Solution
(MARIE) :**

```
                                ORG 100

Main,                          Load   Main_ArrayBaseAddress
                               Store
TestPrimeArray ArrayBaseAddress
                               Load   Main_ArrayNumCells
                               Store   TestPrimeArray ArrayNumCells
                               JnS     TestPrimeArray
                               Halt

Main_Array,                    Dec 1
                               Dec 2
                               Dec 3
                               Dec 4
                               Dec 5

Main_ArrayBaseAddress,        Hex 106 / address of a[0]
Main_ArrayNumCells,          Dec 5   / number of array cells
```

Solution (MARIE - cont) :

```

TestPrimeArray,      Hex 0

                                Clear
                                Store   TestPrimeArray ArrayCellIndex    / i <- 0 ; mandatory !

TestPrimeArray Loop,  Load   TestPrimeArray ArrayCellIndex
                        Subt    TestPrimeArray ArrayNumCells
                        Skipcond 000                                / check if i < n
                        Jump     TestPrimeArray EndLoop

                                Load   TestPrimeArray ArrayBaseAddress
                                Add     TestPrimeArray ArrayCellIndex
                                Store   TestPrimeArray ArrayCellAddress / new address of a[i]

                                LoadI  TestPrimeArray ArrayCellAddress / load a[i]
                                Store   Prime_N
                                JnS     Prime
                                Load   Prime_P
                                Output   / outputs Prime(a[i])

                                Load   TestPrimeArray ArrayCellIndex
                                Add     One
                                Store   TestPrimeArray ArrayCellIndex    / i++

                                Jump     TestPrimeArray Loop

TestPrimeArray EndLoop, JumpI  TestPrimeArray

TestPrimeArray ArrayBaseAddress, Hex 0    / address of a[0]; input parameter
TestPrimeArray ArrayNumCells,   Dec 0    / number of array cells; input parameter

TestPrimeArray ArrayCellAddress, Hex 0    / address of a[i]
TestPrimeArray ArrayCellIndex,   Dec 0    / index i

One,                               Dec 1    / constant 1

```

```

Prime,      Hex 0
            / ...
Prime_EndLoop, JumpI Prime

Prime_N,    Dec    0
Prime_P,    Dec    0
Prime_I,    Dec    0
One,        Dec    1

```


MC13 Write the following subroutines in MARIE assembly (and small programs that use them):

- a) A subroutine “MakeSymArray” that produces an array of integers that are symmetric to the integers of another array.
- b) A subroutine “MakelsPrimeArray” that produces an array whose cells will be 1 (= True) or 0 (= False) if the corresponding cells of another array of integers contain a prime number, or not.

Note: in the C and MARIE code of these exercises, in addition to using the subroutine names as a prefix for tags and variable names, the suffixes Input and Output are also used for variables, to tell if they are used as input parameters or to return outputs.

MC13 Write the following subroutines in MARIE assembly (and small programs that use them):

- a) A subroutine “MakeSymArray” that produces an array of integers that are symmetric to the integers of another array.

Solution (C):

```
int Sym (int Sym_Input) { // see MC11.c) }

void MakeSymArray (int MakeSymArray_Array1[], int MakeSymArray_Array2[],
                  int MakeSymArray_ArrayNumCells)
{
    for (int i=0; i<MakeSymArray_ArrayNumCells; i++)
        MakeSymArray_Array2[i] = Sym(MakeSymArray_Array1[i]);
}

main()
{
    int Main_Array1[5]={1,2,3,4,5}, Main_Array2[5], Main_ArrayNumCells=5;
    MakeSymArray(Main_Array1, Main_Array2, Main_ArrayNumCells);
}
```

MC13 Write the following subroutines in MARIE assembly (and small programs that use them):

Solution (MARIE):

```
                                ORG 100

Main,                          Load   Main_ArrayBaseAddress1
                               Store   MakeSymArray_ArrayBaseAddress1
                               Load    Main_ArrayBaseAddress2
                               Store    MakeSymArray_ArrayBaseAddress2
                               Load    Main_ArrayNumCells
                               Store    MakeSymArray_ArrayNumCells
                               JnS      MakeSymArray
                               Halt

Main_Array1,                   Dec 1
                               Dec 2
                               Dec 3
                               Dec 4
                               Dec 5

Main_Array2,                   Dec 0
                               Dec 0
                               Dec 0
                               Dec 0
                               Dec 0

Main_ArrayBaseAddress1, Hex 108 / address of a[0]
Main_ArrayBaseAddress2, Hex 10D / address of b[0]
Main_ArrayNumCells,      Dec 5   / number of array cells
```

MC13

.a)

```
Sym,      Hex 0
          Clear
          Subt Sym_Input
          Store Sym_Output
          JumpI Sym

Sym_Input, Dec 0
Sym_Output, Dec 0
```

```
MakeSymArray,      Hex 0

                                Clear
                                Store  MakeSymArray_ArrayCellIndex    / i <- 0 ; mandatory !

MakeSymArray_Loop,  Load  MakeSymArray_ArrayCellIndex
                    Subt   MakeSymArray_ArrayNumCells
                    Skipcond 000                                / check if i < n
                    Jump   MakeSymArray_EndLoop

Sym,      Hex 0
          Clear
          Subt Sym_Input
          Store Sym_Output
          JumpI Sym

                                Load  MakeSymArray_ArrayBaseAddress1
                                Add    MakeSymArray_ArrayCellIndex
                                Store  MakeSymArray_ArrayCellAddress1 / new address of a[i]

                                LoadI  MakeSymArray_ArrayCellAddress1 / load a[i]
                                Store  Sym_Input
                                JnS    Sym

                                Load  MakeSymArray_ArrayBaseAddress2
                                Add    MakeSymArray_ArrayCellIndex
                                Store  MakeSymArray_ArrayCellAddress2 / new address of b[i]

                                Load  Sym_Output                                / load Sym(a[i])
                                StoreI MakeSymArray_ArrayCellAddress2 / store in b[i]

                                Load  MakeSymArray_ArrayCellIndex
                                Add    One
                                Store  MakeSymArray_ArrayCellIndex    / i++

                                Jump   MakeSymArray_Loop

MakeSymArray_EndLoop, JumpI  MakeSymArray
```

**Solution
(MARIE
- cont.):**



Solution (MARIE - cont.):

```
MakeSymArray_ArrayBaseAddress1, Hex 0    / address of a[0]; input parameter
MakeSymArray_ArrayBaseAddress2, Hex 0    / address of b[0]; input parameter
MakeSymArray_ArrayNumCells,      Dec 0    / number of array cells; input parameter

MakeSymArray_ArrayCellAddress1, Hex 0    / address of a[i]
MakeSymArray_ArrayCellAddress2, Hex 0    / address of b[i]
MakeSymArray_ArrayCellIndex,     Dec 0    / index i

One,                             Dec 1    / constant 1
```

MATC1 – Consolidated Exercise

(Assemblage, Tracing and Coding)

1. Consider the following program in MARIE assembly:

```

Loop,      ORG 100
           Load Counter
           Subt Num
           Skipcond 800
           Jump LoopBody
           Jump LoopEnd
LoopBody,  Load Counter
           Add One
           Store Counter
           Jump Loop
LoopEnd,   Halt
Counter,   Dec 2
Num,       Dec 10
One,       Dec 1
```

Opcode	Instruction
1	<u>Load</u> X
2	<u>Store</u> X
3	<u>Add</u> X
4	<u>Subt</u> X
5	Input
6	Output
7	<u>Halt</u>
8	<u>Skipcond</u> X X=000: <u>AC</u> <0? X=400: <u>AC</u> ==0? X=800: <u>AC</u> >0?

MARIE
Instruction Set

Opcode	Instruction
9	<u>Jump</u> X
0	<u>Jns</u> X
A	<u>Clear</u>
B	<u>AddI</u> X
C	<u>JumpI</u> X
D	<u>LoadI</u> X
E	<u>StoreI</u> X

1.1 Assemble the program and fill in the following tables
(note: all tables have the exact number of lines needed)

MATC1 – Consolidated Exercise

(Assemblage, Tracing and Coding)



1.1.a The Symbol Table that results from the 1st stage of assemblage (fill the table by the order in which the symbols are found in the source code during the assemblage algorithm; show the addresses in hexadecimal):

Symbol Table

Symbol	Address

MATC1 – Consolidated Exercise

(Assemblage, Tracing and Coding)

1.1.a The Symbol Table that results from the 1st stage of assemblage (fill the table by the order in which the symbols are found in the source code during the assemblage algorithm; show the addresses in hexadecimal):

Answer:

Symbol Table

Symbol	Address
Loop	100
Counter	10A
Num	10B
LoopBody	105
LoopEnd	109
One	10C

MATC1 – Consolidated Exercise

(Assemblage, Tracing and Coding)

1.1.b List of the addresses of the memory positions used, and its content (final result of the assemblage), in hexadecimal:

Address	Content

MATC1 – Consolidated Exercise

(Assemblage, Tracing and Coding)

1.1.b List of the addresses of the memory positions used, and its content (final result of the assemblage), in hexadecimal:

Answer:

Address	Content
100	1 10A
101	4 10B
102	8 800
103	9 105
104	9 109
105	1 10A
106	3 10C
107	2 10A
108	9 100
109	7 000
10A	0 002
10B	0 00A
10C	0 001

MATC1 – Consolidated Exercise

(Assemblage, Tracing and Coding)

2. Based on the answer to question 1.1.b), trace the first five instructions effectively executed by the program. Then, in the table bellow, present each instruction (Inst1 to Inst5, by the order of execution) and the final value (in hexadecimal) of the registers MAR, MBR, PC, IR e AC, right after the execution of that instruction:

Instruction / CPU Register	Inst1:	Inst2:	Inst3:	Inst4:	Inst5:
MAR:					
MBR:					
PC:					
IR:					
AC:					

MATC1 – Consolidated Exercise

(Assemblage, Tracing and Coding)

2. Based on the answer to question 1.1.b), trace the first five instructions effectively executed by the program. Then, in the table bellow, present **each instruction** (Inst1 to Inst5, by the order of execution) and the **final value** (in hexadecimal) of the registers MAR, MBR, PC, IR e AC, right after the execution of that instruction:

Answer:

Instruction / CPU Register	Inst1: Load Counter	Inst2: Subt Num	Inst3: SkipCond 800	Inst4: Jump LoopBody	Inst5: Load Counter
MAR:	100, 10A	101, 10B	102, 800	103, 105	105, 10A
MBR:	110A, 0002	410B, 000A	8800	9105	110A, 0002
PC:	100, 101	101, 102	102, 103	103,104, 105	105, 106
IR:	110A	410B	8800	9105	110A
AC:	0002	FFF8	FFF8	FFF8	0002

MATC1 – Consolidated Exercise

(Assemblage, Tracing and Coding)

3. The numbers of Lucas are generated by the formula $L_n = L_{n-2} + L_{n-1}$, with $L_0 = 2$ and $L_1 = 1$, and each of the next numbers being equal to the sum of the two previous Lucas numbers. For instance, the first 5 Lucas numbers are: 2, 1, 3 (=1+2), 4 (=3+1), 7 (=4+3). The C program bellow has a **Lucas** function that returns the N'th Lucas number and a **main** function that makes use of it (note: assume $N \geq 0$ always).

```
int main()
{
    int N, L;

    scanf("%d", &N);

    if (N < 2)
        L = 2 - N;
    else
        L = Lucas(N);

    printf("%d", L);
}
```

```
int Lucas(int Num)
{
    int Number0, Number1, Counter, Current;
    Number0 = 2; Number1 = 1;

    for (Counter = 2; Counter <= Num; Counter++) {
        Current = Number1 + Number0;
        Number0 = Number1;
        Number1 = Current;
    }

    return Current;
}
```

MATC1 – Consolidated Exercise (Assemblage, Tracing and Coding)

3.a) Present the MARIE assembly code of function **main**.

```
int main()  
{  
    int N, L;  
  
    scanf("%d", &N);  
  
    if (N < 2)  
        L = 2 - N;  
    else  
        L = Lucas(N);  
  
    printf("%d", L);  
}
```

```
int Lucas(int Num)  
{  
    int Number0, Number1, Counter, Current;  
    Number0 = 2; Number1 = 1;  
  
    for (Counter = 2; Counter <= Num; Counter++) {  
        Current = Number1 + Number0;  
        Number0 = Number1;  
        Number1 = Current;  
    }  
  
    return Current;  
}
```

MATC1 – Consolidated Exercise (Assemblage, Tracing and Coding)

3.a) Present the MARIE assembly code of function **main**.

Answer:

```
ORG 100
Input
Store N
// Load N
Subt Two
If, Skipcond 000
Jump Else
Then, Load Two
Subt N
Store L
Jump EndIf
Else, Load N
Store Num
JnS Lucas
Load Current
Store L
EndIf, Load L
Output
Halt

N, Dec 0
L, Dec 0
Two, Dec 2
```

MATC1 – Consolidated Exercise (Assemblage, Tracing and Coding)

3.b) Present the MARIE assembly code of function **Lucas**.

```
int main()  
{  
    int N, L;  
  
    scanf("%d", &N);  
  
    if (N < 2)  
        L = 2 - N;  
    else  
        L = Lucas(N);  
  
    printf("%d", L);  
}
```

```
int Lucas(int Num)  
{  
    int Number0, Number1, Counter, Current;  
    Number0 = 2; Number1 = 1;  
  
    for (Counter = 2; Counter <= Num; Counter++) {  
        Current = Number1 + Number0;  
        Number0 = Number1;  
        Number1 = Current;  
    }  
  
    return Current;  
}
```


MATC1 – Consolidated Exercise (Assemblage, Tracing and Coding)

3.b) Present the MARIE assembly code of function **Lucas**.

Answer:

```
Lucas,      Hex 0
            Load Two
            Store Number0 // Number0 = 2
            Load One
            Store Number1 // Number1 = 1
            Load Two
            Store Counter // Counter = 2
Loop,       Load Counter
            Subt Num
            Skipcond 800 // Counter > Num ?
            Jump LoopBody
            Jump LoopEnd // or JumpI Lucas
LoopBody,   Load Number1
            Add Number0
            Store Current // Current = Number1 + Number0
            Load Number1
            Store Number0 // Number0 = Number1
            Load Current
            Store Number1 // Number1 = Current
            Load Counter
            Add One
            Store Counter // Counter ++
            Jump Loop
LoopEnd,    JumpI Lucas
```

```
Num,      Dec 0
One,      Dec 1
Two,      Dec 2
Counter,  Dec 0
Current,  Dec 0
Number1,  Dec 0
Number0,  Dec 0
```